

Universidade do Minho

Licenciatura em Ciências da Computação

Unidade Curricular de Otimização

Ano Letivo 2025/2026

T3 - Método de Newton BFGS

Relatório de Trabalho Prático

Grupo 6

A101777 - Gustavo Alves de Araújo Gomes

A102935 - Ivo Costa Sousa

A108579 - João Carlos Teixeira Neiva

A102528 - João Pedro Ribeiro Matos

A109069 - Miguel Ângelo Oliveira da Silva

A104244 - Miguel Dinis Páscoa

A102878 - Ricardo Eusébio Cerqueira

Introdução

Neste trabalho foi implementado o método **BFGS (Broyden–Fletcher–Goldfarb–Shanno)** para a resolução de problemas de otimização não linear sem restrições.

O método BFGS pertence à classe dos métodos **Quasi-Newton**, cujo principal objetivo é aproximar a matriz Hessiana da função objetivo sem necessidade de calcular explicitamente segundas derivadas. Desta forma, obtém-se um método significativamente mais eficiente do que o método de Newton clássico, especialmente em problemas de maior dimensão.

Para garantir convergência global, o comprimento de passo foi determinado através de uma **procura linear de Armijo com backtracking**.

Além disso, sempre que a condição de curvatura não é satisfeita, foi utilizada uma estratégia de **Damped BFGS Update**, permitindo preservar a positividade definida da aproximação da Hessiana inversa.

Formulação do Problema

Pretende-se resolver o seguinte problema de otimização:

$$\min_{w \in \mathbb{R}^n} F(w)$$

onde:

- $F : \mathbb{R}^n \rightarrow \mathbb{R}$ é continuamente diferenciável;
 - w representa o vetor de variáveis em n dimensões.
-

Método BFGS

Ideia Geral

O método BFGS constrói iterativamente uma aproximação da inversa da matriz Hessiana da função objetivo.

Em cada iteração, é calculada uma direção de descida dada por:

$$p_k = -H_k \nabla F(w_k)$$

onde:

- H_k representa a aproximação da inversa da Hessiana na iteração k ;
- $\nabla F(w_k)$ é o gradiente da função no ponto atual w_k .

O novo ponto é obtido através de:

$$w_{k+1} = w_k + \eta_k p_k$$

onde:

- η_k é o comprimento de passo obtido pela procura linear de Armijo.

Procura Linear de Armijo com Backtracking

Objetivo

A procura linear tem como objetivo encontrar um comprimento de passo adequado que produza uma diminuição suficiente no valor da função objetivo.

Para tal, é avaliada a condição de Armijo:

$$F(w_k + \eta_k p_k) \leq F(w_k) + c_1 \eta_k \nabla F(w_k)^T p_k$$

com os parâmetros de projeto definidos por:

- $c_1 = 10^{-4}$
- $\eta_0 = 1$

Algoritmo de Backtracking

O procedimento inicia-se com o passo máximo unitário:

$$\eta_k = 1$$

Caso a condição de Armijo não seja satisfeita, o passo é reduzido sucessivamente através do fator de contração β :

$$\eta_k \leftarrow \beta \eta_k$$

com:

$$0 < \beta < 1$$

até que a condição de decréscimo suficiente seja verificada.

Atualização BFGS

Após o cálculo do novo ponto de iteração, definem-se os vetores de variação de posição (s_k) e variação de gradiente (y_k):

$$s_k = w_{k+1} - w_k$$

$$y_k = \nabla F(w_{k+1}) - \nabla F(w_k)$$

A atualização da aproximação da Hessiana inversa é dada por:

$$H_{k+1} = (I - \rho_k s_k y_k^T) H_k (I - \rho_k y_k s_k^T) + \rho_k s_k s_k^T$$

onde ρ_k é um escalar de escala definido por:

$$\rho_k = \frac{1}{y_k^T s_k}$$

Esta formulação permite obter uma aproximação robusta da inversa da Hessiana utilizando unicamente informações de primeira ordem (gradientes).

Condição de Curvatura

Para garantir que a matriz H_{k+1} permaneça definida positiva a cada iteração, é teoricamente necessário verificar a condição de curvatura:

$$s_k^T y_k > 0$$

Caso esta condição não seja satisfeita, a atualização BFGS convencional pode perder estabilidade numérica ou comprometer a propriedade de descida da direção calculada.

Estratégia de Amortecimento - *Damped BFGS Update*

Quando a condição de curvatura $s_k^T y_k > 0$ não é adequadamente satisfeita, a atualização standard do BFGS pode originar a perda de positividade definida de H_k . Para contornar este problema, aplica-se a estratégia de amortecimento (*Damped BFGS Update*) descrita na Secção 18.3 de Nocedal & Wright.

Define-se o parâmetro de amortecimento θ_k :

$$\theta_k = \begin{cases} 1, & \text{se } s_k^T y_k \geq 0.2 s_k^T B_k s_k \\ \frac{0.8 s_k^T B_k s_k}{s_k^T B_k s_k - s_k^T y_k}, & \text{caso contrário} \end{cases}$$

onde $B_k = H_k^{-1}$. Em seguida, substitui-se o vetor de variação do gradiente original y_k pelo vetor modificado e amortecido r_k :

$$r_k = \theta_k y_k + (1 - \theta_k) B_k s_k$$

A atualização da matriz H_{k+1} passa a utilizar o vetor r_k em substituição de y_k , assegurando forçosamente que $s_k^T r_k > 0$, preservando assim a positividade definida da Hessiana inversa em todas as iterações do algoritmo.

Critério de Paragem

O algoritmo iterativo é interrompido sempre que a norma do infinito do gradiente for inferior ou igual à tolerância estabelecida:

$$\|\nabla F(w_k)\|_\infty \leq 10^{-6}$$

ou quando é atingido o número máximo admissível de iterações:

$$K_{\max} = 1000$$

Informação Registada

Em cada iteração efetuada pelo algoritmo, os seguintes dados numéricos foram armazenados para monitorização:

Iteração (k)	Vetor ($w^{(k)}$)	Norma Gradiente ($ \nabla F(w^{(k)}) _{\infty}$)	Passo (η_k)	Custo ($F(w^{(k)})$)
------------------	---------------------	--	--------------------	------------------------

Este registo de dados estruturado possibilita avaliar de forma rigorosa a velocidade de convergência do método, a evolução temporal da função objetivo, o comportamento do comprimento de passo adaptativo e o decréscimo assintótico da norma do gradiente.

Casos de Estudo e Resultados por Problema

Para avaliar o desempenho, a robustez e a convergência do método BFGS implementado, considerou-se o conjunto de problemas de otimização não linear propostos na alínea 7(e) da Ficha 5.

Problema 1 (*Função de Rosenbrock*)

Função objetivo:

$$F_1(w) = 100(w_2 - w_1^2)^2 + (1 - w_1)^2$$

Ponto inicial: $w^{(0)} = [-1.2, 1]^T$ **Mínimo global conhecido:** $w^* = [1, 1]^T$, $F^* = 0$

Resultados:

Método	w^*	F^*	Iterações	$ \nabla F _{\infty}$ final
BFGS próprio	(1.0000, 1.0000)	2.75×10^{-17}	34	$< 10^{-6}$
fminunc MATLAB	(1.0000, 1.0000)	9.04×10^{-13}	36	1.65×10^{-5}

Evolução da função objetivo (BFGS):

Iteração	w_1	w_2	$F(w^{(k)})$	$ \nabla F _{\infty}$	η_k
0	-1.2000	1.0000	24.2000	215.60	0.0010
5	-0.4976	0.2673	2.2816	3.94	1.0000
10	-0.0312	-0.0311	1.1663	6.42	1.0000
20	0.6035	0.3613	0.1580	0.58	1.0000
30	0.9979	0.9953	0.0000	0.18	1.0000

Iteração	w_1	w_2	$F(w^{(k)})$	$ \nabla F _\infty$	η_k
34	1.0000	1.0000	2.75×10^{-17}	$< 10^{-6}$	1.0000

Análise:

A função de Rosenbrock é o problema de teste mais exigente deste conjunto. Trata-se de uma função **não convexa**, com um vale curvo e estreito ao longo da parábola $w_2 = w_1^2$, onde o mínimo global se encontra. A convergência é lenta para qualquer método de gradiente.

Na primeira iteração, o gradiente $\|\nabla F\|_\infty = 215.6$ obrigou o backtracking a reduzir o passo até $\eta_0 = 0.001$. À medida que o iterado se aproximou do fundo do vale, o passo estabilizou em $\eta_k = 1$, o que indica que a direção BFGS se tornou cada vez mais precisa. O BFGS próprio convergiu em **34 iterações** com $F^* = 2.75 \times 10^{-17}$, superando ligeiramente o `fminunc` (36 iterações, $F^* = 9.04 \times 10^{-13}$). Ambos encontram o mínimo global correto, mas o BFGS próprio atingiu uma precisão ligeiramente superior.

Problema 2

Função objetivo:

$$F_2(w) = 4w_1^2 + 2w_2^2 + 4w_1w_2 - 3w_1$$

Ponto inicial: $w^{(0)} = [2, 2]^T$

Resultados:

Método	w^*	F^*	Iterações
BFGS próprio	(0.7500, -0.7500)	-1.1250	7
fminunc MATLAB	(0.7500, -0.7500)	-1.1250	7

Evolução da função objetivo (BFGS):

Iteração	w_1	w_2	$F(w^{(k)})$	$ \nabla F _\infty$	η_k
0	2.0000	2.0000	34.0000	21.00	0.1250
1	-0.6250	0.0000	3.4375	8.00	1.0000
3	0.7337	-0.6588	-1.1132	0.30	1.0000
5	0.7502	-0.7502	-1.1250	0.001	1.0000
7	0.7500	-0.7500	-1.1250	$< 10^{-6}$	1.0000

Análise:

Convergência **idêntica** entre os dois métodos — ambos em 7 iterações com o mesmo valor ótimo. A função é suave e bem condicionada, permitindo que o BFGS construa rapidamente uma boa aproximação da Hessiana. O passo unitário é aceite a partir da iteração 1, confirmando que a direção de procura é de alta qualidade desde cedo.

Problema 3

Função objetivo:

$$F_3(w) = w_1^2 + 2w_2^2 - 2w_1w_2 - 2w_2$$

Ponto inicial: $w^{(0)} = [0, 0]^T$

Resultados:

Método	w^*	F^*	Iterações
BFGS próprio	(1.0000, 1.0000)	-1.0000	2
fminunc MATLAB	(1.0000, 1.0000)	-1.0000	3

Evolução da função objetivo (BFGS):

Iteração	w_1	w_2	$F(w^{(k)})$	$ \nabla F _\infty$	η_k
0	0.0000	0.0000	0.0000	2.00	0.2500
1	0.0000	0.5000	-0.5000	1.00	1.0000
2	1.0000	1.0000	-1.0000	0.00	1.0000

Análise:

Convergência muito rápida — o BFGS converge em apenas **2 iterações**, contra 3 do `fminunc`. Na iteração 2, o gradiente é exatamente zero ($\|\nabla F\|_\infty = 0$), indicando que o método chegou ao mínimo exato. Trata-se de uma função simples e bem comportada.

Problema 4

Função objetivo:

$$F_4(w) = (w_1 + w_2)^4 + w_2^2$$

Ponto inicial: $w^{(0)} = [2, -2]^T$

Resultados:

Método	w^*	F^*	Iterações
BFGS próprio	(-0.0039, -0.0000)	2.26×10^{-10}	19
fminunc MATLAB	(0.0090, -0.0000)	6.68×10^{-9}	18

Evolução da função objetivo (BFGS):

Iteração	w_1	w_2	$F(w^{(k)})$	$ \nabla F _\infty$	η_k
0	2.0000	-2.0000	4.0000	4.00	0.2500
1	2.0000	-1.0000	2.0000	4.00	1.0000
5	-0.1659	-0.0068	0.0009	0.034	1.0000
10	-0.0476	0.0007	0.0000	0.001	1.0000
19	-0.0039	-0.0000	2.26×10^{-10}	$< 10^{-6}$	1.0000

Análise:

O BFGS e o `fminunc` encontram soluções ligeiramente diferentes ($w_1 \approx -0.004$ vs $w_1 \approx 0.009$), mas com valores de função objetivo muito próximos de zero. Isto sugere que existe uma família de soluções próximas do ótimo (curvatura muito fraca perto do mínimo). O BFGS precisou de 19 iterações contra 18 do `fminunc`, com uma precisão ligeiramente superior ($F^* = 2.26 \times 10^{-10}$ vs 6.68×10^{-9}). Os passos $\eta_k = 1$ constantes indicam que o método não encontrou dificuldades de curvatura.

Problema 5

Função objetivo:

$$F_5(w) = \frac{1}{2}(2w_1^2 + 3w_2^2 + 4w_3^2) + 8w_1 + 9w_2 + 8w_3$$

Ponto inicial: $w^{(0)} = [0, 0, 0]^T$ | **Dimensão:** $n = 3$

Resultados:

Método	w^*	F^*	Iterações
BFGS próprio	(-4.0000, -3.0000, -2.0000)	-37.5000	11
fminunc MATLAB	(-4.0000, -3.0000, -2.0000)	-37.5000	6

Evolução da função objetivo (BFGS):

Iteração	w_1	w_2	w_3	$F(w^{(k)})$	$ \nabla F _\infty$
0	0.000	0.000	0.000	0.000	9.00
1	-4.000	-4.500	-4.000	-26.125	8.00
3	-3.818	-1.723	-2.197	-34.944	3.83
7	-3.998	-3.003	-2.002	-37.500	0.009
11	-4.000	-3.000	-2.000	-37.500	$< 10^{-6}$

Análise:

Ambos os métodos encontram o mesmo mínimo correto, mas o **fminunc** convergiu em **6 iterações** contra **11 do BFGS próprio**. Esta diferença é mais pronunciada em 3D e deve-se ao **escalonamento da matriz inicial** H_0 .

O **fminunc** escala automaticamente H_0 após o primeiro passo (usando a heurística $H_0 \leftarrow \frac{y^{(0)T} s^{(0)}}{y^{(0)T} y^{(0)}} I$), enquanto o BFGS próprio mantém $H_0 = I$ fixo. Em 3 dimensões, a matriz identidade é uma aproximação inicial mais pobre da Hessiana real, exigindo mais iterações para convergir.

Problema 6

Função objetivo:

$$F_6(w) = \frac{1}{2}(5w_1^2 + 7w_2^2 + 9w_3^2 + 4w_1w_2 + 2w_1w_3 + 6w_2w_3) + 9w_1 + 8w_3$$

Ponto inicial: $w^{(0)} = [0, 0, 0]^T$ | **Dimensão:** $n = 3$

Resultados:

Método	w^*	F^*	Iterações
BFGS próprio	(-3.8075, 1.5021, -0.9665)	-36.2301	9
fminunc MATLAB	(-3.8075, 1.5021, -0.9665)	-36.2301	8

Evolução da função objetivo (BFGS):

Iteração	w_1	w_2	w_3	$F(w^{(k)})$	$ \nabla F _\infty$
0	0.000	0.000	0.000	0.000	17.00
2	-5.950	2.232	-1.059	-25.980	9.35
5	-3.971	1.515	-0.962	-36.168	0.785
8	-3.808	1.502	-0.967	-36.230	0.0000
9	-3.808	1.502	-0.967	-36.230	$< 10^{-6}$

Análise:

Resultados muito próximos — apenas 1 iteração de diferença (9 vs 8). Ambos encontram exatamente o mesmo mínimo local. A função tem estrutura não quadrática em 3D mas é bem comportada, permitindo convergência rápida. O BFGS próprio apresenta uma iteração extra que se deve à menor qualidade inicial de $H_0 = I$ comparada com o escalonamento adaptativo do **fminunc**.

Problema 7

Função objetivo:

$$F_7(w) = w_1^2 + w_2^2 + w_3^2$$

Ponto inicial: $w^{(0)} = [1, 1, 1]^T$ | **Dimensão:** $n = 3$

Resultados:

Método	w^*	F^*	Iterações
BFGS próprio	(0, 0, 0)	0.0000	1
fminunc MATLAB	(0, 0, 0)	0.0000	1

Evolução da função objetivo (BFGS):

Iteração	w_1	w_2	w_3	$F(w^{(k)})$	$ \nabla F _\infty$
0	1.000	1.000	1.000	3.000	2.00
1	0.000	0.000	0.000	0.000	0.00

Análise:

Convergência em **1 única iteração** por ambos os métodos. Este resultado indica que a função é **quadrática** e $H_0 = I$ é proporcional à Hessiana exata. Com uma função quadrática $F(w) = \frac{1}{2}w^T Q w + b^T w + c$, o método de Newton (e portanto o BFGS quando $H_k \approx Q^{-1}$) converge exatamente em 1 passo, como previsto pela teoria de otimização.

Problema 8 (*Efeito do Parâmetro de Penalidade α*)

Função objetivo:

$$F_8(w) = (w_1 - 1)^2 + (w_2 - 1)^2 + \alpha (w_1^2 + w_2^2 - 0.25)^2$$

Ponto inicial: $w^{(0)} = [1, -1]^T$ **Parâmetros testados:** $\alpha \in \{1, 10, 100\}$

O termo $\alpha(w_1^2 + w_2^2 - 0.25)^2$ é uma **função de penalidade** que penaliza soluções afastadas da circunferência $w_1^2 + w_2^2 = 0.25$. À medida que α aumenta, o problema torna-se mais difícil (maior condicionamento da Hessiana).

Resultados — $\alpha = 1$:

Método	w^*	F^*	Iterações	$ \nabla F _\infty$ inicial
BFGS próprio	(0.5641, 0.5641)	0.5293	9	11.0
fminunc MATLAB	(0.5641, 0.5641)	0.5293	8	11.0

Resultados — $\alpha = 10$:

Método	w^*	F^*	Iterações	$ \nabla F _\infty$ inicial
BFGS próprio	(0.4026, 0.4026)	0.7688	12	74.0

Método	w^*	F^*	Iterações	$ \nabla F _\infty$ inicial
fminunc MATLAB	(0.4026, 0.4026)	0.7688	8	74.0

Resultados — $\alpha = 100$:

Método	w^*	F^*	Iterações	$ \nabla F _\infty$ inicial
BFGS próprio	(0.3598, 0.3598)	0.8277	22	704.0
fminunc MATLAB	(0.3598, 0.3598)	0.8277	8	704.0

Evolução das iterações com $\alpha = 100$ (BFGS):

Iteração	w_1	w_2	$F(w^{(k)})$	$ \nabla F _\infty$	η_k
0	1.0000	-1.0000	310.25	704.0	0.0020
5	0.0481	0.5638	1.5886	14.95	1.0000
10	0.1513	0.4914	0.9995	1.80	1.0000
15	0.3205	0.3976	0.8363	0.51	1.0000
20	0.3598	0.3598	0.8277	0.003	1.0000
22	0.3598	0.3598	0.8277	$< 10^{-6}$	1.0000

Análise — Efeito de α :

α	w^*	F^*	Iter BFGS	Iter fminunc	Razão
1	(0.564, 0.564)	0.5293	9	8	$1.1 \times$
10	(0.403, 0.403)	0.7688	12	8	$1.5 \times$
100	(0.360, 0.360)	0.8277	22	8	$2.75 \times$

O efeito de α é claro: à medida que α aumenta, a solução ótima aproxima-se progressivamente da circunferência de raio 0.5 (note que $\sqrt{2} \times 0.360 \approx 0.509 \approx 0.5$), como seria de esperar de uma função de penalidade.

O número de iterações do BFGS próprio cresce significativamente com α ($9 \rightarrow 12 \rightarrow 22$), enquanto o **fminunc** mantém 8 iterações independentemente de α . Esta diferença deve-se a dois fatores:

- Mau condicionamento crescente:** com $\alpha = 100$, o gradiente inicial é $\|\nabla F\|_\infty = 704$, tornando $H_0 = I$ uma péssima aproximação inicial da Hessiana inversa.
- Procura linear:** o **fminunc** usa as **condições de Wolfe** (Armijo + curvatura), que garantem automaticamente $s_k^T y_k > 0$ e produzem passos de melhor qualidade. O BFGS próprio usa apenas Armijo, compensando com o Damped update, mas necessitando de mais iterações.

Tabela Resumo Geral

Prob.	dim	$w^{(0)}$	w^*	F^* (BFGS)	F^* (fminunc)	Iter BFGS	Iter fminunc	Vencedor
P1 (Rosenbrock)	2	$[-1.2, 1]$	$(1, 1)$	2.75×10^{-17}	9.04×10^{-13}	34	36	BFGS
P2	2	$[2, 2]$	$(0.75, -0.75)$	-1.1250	-1.1250	7	7	Igual
P3	2	$[0, 0]$	$(1, 1)$	-1.0000	-1.0000	2	3	BFGS
P4	2	$[2, -2]$	$(\approx 0, 0)$	2.26×10^{-10}	6.68×10^{-9}	19	18	fminunc
P5	3	$[0, 0, 0]$	$(-4, -3, -2)$	-37.5000	-37.5000	11	6	fminunc
P6	3	$[0, 0, 0]$	$(-3.808, 1.502, -0.967)$	-36.2301	-36.2301	9	8	fminunc
P7	3	$[1, 1, 1]$	$(0, 0, 0)$	0.0000	0.0000	1	1	Igual
P8 $\alpha = 1$	2	$[1, -1]$	$(0.564, 0.564)$	0.5293	0.5293	9	8	fminunc
P8 $\alpha = 10$	2	$[1, -1]$	$(0.403, 0.403)$	0.7688	0.7688	12	8	fminunc
P8 $\alpha = 100$	2	$[1, -1]$	$(0.360, 0.360)$	0.8277	0.8277	22	8	fminunc

Resumo: O BFGS próprio vence ou empata em **4/10** casos; **fminunc** vence em **6/10** casos. Em todos os casos, ambos os métodos convergem para **o mesmo mínimo**.

Discussão de Resultados

Qualidade das Soluções

Em todos os 10 casos testados, o BFGS próprio e o **fminunc** encontraram o **mesmo mínimo**, com valores de função objetivo praticamente idênticos. Esta é a observação mais importante: a **implementação própria é correta** e equivalente ao método de referência do MATLAB em termos de qualidade da solução.

Eficiência Computacional

A principal diferença entre os dois métodos está no **número de iterações necessárias**, que reflete diferenças algorítmicas importantes:

Vantagem do **fminunc**:

- O **fminunc** usa as **condições de Wolfe** (Armijo + condição de curvatura), que garantem $s_k^T y_k > 0$ e produzem atualizações BFGS de maior qualidade.
- O **fminunc** escala automaticamente H_0 após o primeiro passo, usando a heurística:

$$H_0 \leftarrow \frac{y_0^T s_0}{y_0^T y_0} I$$

Esta adaptação é particularmente benéfica em problemas com gradientes muito elevados (P8 com $\alpha = 100$, onde $\|\nabla F\|_\infty = 704$).

Vantagem do BFGS próprio:

- O **Damped BFGS Update** implementado é uma estratégia mais robusta do que a simples verificação da condição de curvatura — em vez de descartar a atualização quando $s_k^T y_k \leq 0$, modifica y_k para garantir positividade definida.
- No problema P1 (Rosenbrock), o BFGS próprio converge em menos iterações (34 vs 36), possivelmente porque o Damped update preservou melhor a informação de curvatura ao longo das iterações.

Taxa de Convergência

Ambos os métodos apresentam **convergência superlinear** na vizinhança do mínimo, como previsto pela teoria Quasi-Newton. Isto é visível na rápida diminuição de $\|\nabla F\|_\infty$ nas últimas iterações de todos os problemas. No Problema 7, a convergência em **1 iteração** confirma que para uma função quadrática convexa, o método de Newton (e o BFGS quando $H_0 \approx Q^{-1}$) converge exatamente em 1 passo a partir de qualquer ponto inicial.

Comportamento do Comprimento de Passo

Em quase todos os problemas, o passo unitário $\eta_k = 1$ é aceite a partir da segunda ou terceira iteração, confirmando que:

- A direção de procura BFGS é uma boa direção de descida.
- A condição de Armijo é facilmente satisfeita quando o método está próximo do mínimo.
- O backtracking (passo reduzido) ocorre principalmente na **primeira iteração**, quando o ponto inicial está longe do mínimo e o gradiente é elevado.

A exceção é o Problema 4, onde os passos alternam entre $\eta_k = 1$ e valores ligeiramente reduzidos, indicando uma função com curvatura irregular perto do mínimo.

Influência de $H_0 = I$

A escolha $H_0 = I$ é a inicialização mais simples mas não necessariamente a mais eficiente. Os resultados mostram que esta escolha é adequada para problemas 2D mas introduz ineficiência em problemas 3D (P5: 11 vs 6 iterações) e em problemas com gradientes muito elevados (P8 $\alpha = 100$: 22 vs 8 iterações).

Comparação Teórica dos Métodos

Propriedades Algorítmicas

Propriedade	BFGS próprio	fminunc MATLAB
Classe	Quasi-Newton	Quasi-Newton
Direção de procura	$p_k = -H_k \nabla F(w_k)$	$p_k = -H_k \nabla F(w_k)$
Procura linear	Armijo backtracking	Condições de Wolfe
Atualização H_k	BFGS + Damped update	BFGS standard
Inicialização H_0	$H_0 = I$ (fixo)	$H_0 = I$ (escalado após 1ª iter.)
Crítério de paragem	$ \nabla F _\infty \leq 10^{-6}$	$ \nabla F \leq 10^{-6}$
Convergência global	Garantida (Armijo)	Garantida (Wolfe)
Taxa de convergência	Superlinear (local)	Superlinear (local)
Custo por iteração	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
Robustez a mau cond.	Moderada (Damped ajuda)	Alta (escalamento interno)

Condições de Armijo vs Wolfe

O BFGS próprio usa apenas a **condição de Armijo** (condição suficiente de descida):

$$F(w_k + \eta_k p_k) \leq F(w_k) + c_1 \eta_k \nabla F(w_k)^T p_k$$

O **fminunc** usa adicionalmente a **condição de curvatura** (segunda condição de Wolfe):

$$\nabla F(w_k + \eta_k p_k)^T p_k \geq c_2 \nabla F(w_k)^T p_k$$

A condição de Wolfe garante que $s_k^T y_k > 0$, preservando automaticamente a positividade definida de H_k sem necessidade do Damped update. Isto explica a maior eficiência do **fminunc** em problemas mal condicionados.

Condições de Otimalidade Verificadas

Em todos os problemas, ambos os métodos terminam quando $\|\nabla F(w^*)\|_\infty \leq 10^{-6}$, verificando a **condição necessária de primeira ordem** para um mínimo local:

$$\nabla F(w^*) = 0$$

Esta é a condição para problemas sem restrições, equivalente a afirmar que w^* é um **ponto estacionário** da função objetivo.

Conclusões

O método BFGS revelou-se altamente eficiente na resolução dos problemas de otimização considerados, apresentando taxas de convergência rápidas sem incorrer no custo computacional de calcular explicitamente a matriz Hessiana da função objetivo. A integração da procura linear de Armijo garantiu estabilidade numérica

e convergência global ao algoritmo, enquanto o mecanismo de salvaguarda do **Damped BFGS Update** assegurou a manutenção correta da positividade definida da aproximação da Hessiana inversa.

Comparativamente ao método de Newton clássico, o BFGS reduz drasticamente o custo por iteração mantendo uma elevada robustez prática. Os resultados numéricos obtidos e a comparação com o MATLAB permitem traçar as seguintes conclusões consolidadas:

1. **Correção da implementação:** O BFGS implementado encontra o mesmo mínimo que o `fminunc` do MATLAB em todos os 10 casos testados, confirmando que a implementação desenvolvida é rigorosa e está matematicamente correta.
2. **Eficiência comparativa:** O `fminunc` é ligeiramente mais eficiente na maioria dos problemas (6/10), devido principalmente ao escalonamento adaptativo de H_0 e às condições de Wolfe na procura linear. No entanto, a diferença é marginal para funções mais simples ou de baixa dimensão.
3. **Ponto forte do BFGS próprio:** O **Damped BFGS Update** é uma estratégia manifestamente mais robusta do que simplesmente descartar atualizações quando a condição de curvatura falha, permitindo que o método continue a adaptar a matriz H_k mesmo em regiões de curvatura indefinida.
4. **Impacto da penalidade α (Problema 8):** O aumento de α degrada significativamente o desempenho da implementação fixa (9 para 22 iterações) mas não afeta o `fminunc`. Isto comprova de forma prática a elevada importância do escalonamento da matriz Hessiana inicial face a problemas mal condicionados.